



Sukkur Institute of Business Administration University

Department of Computer Science

Object Oriented Programming using Java

BS – II (CS/AI/SE)

Spring-2024

Lab # 04: Unveiling the World of Classes and Objects, Delving into the Depths of String and Array Manipulation

Instructor: Nimra Mughal

Objectives

After performing this lab, students will be able to understand:

- Operators' precedence and ternary operator
- Intro to classes and objects
 - Static and Instance variables
 - Methods of the class
- Intro to heap and stack memory
- String advanced
 - Immutable Strings
 - Mutable strings (String Buffer and String builder)
 - **String Regular Expressions**
- Input ways
 - Scanner
 - JOptionPane
 - Java Command line arguments
 - **Buffered reader**
- Java Documentation basics
- More practice on arrays

Intro to heap and stack memory

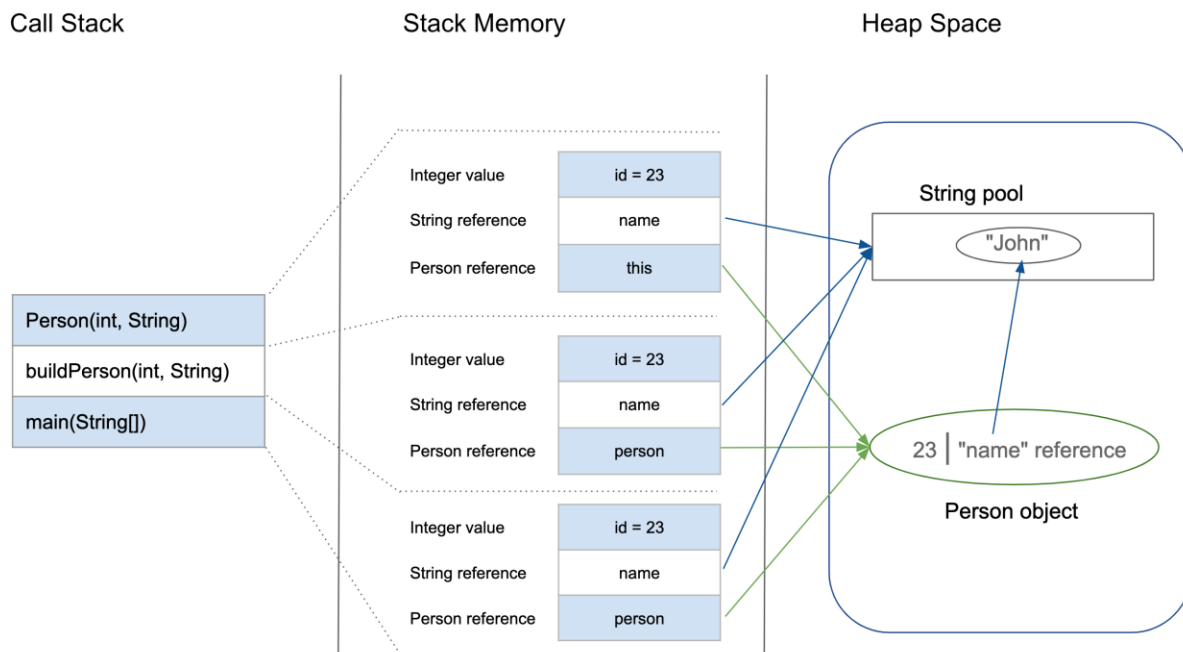
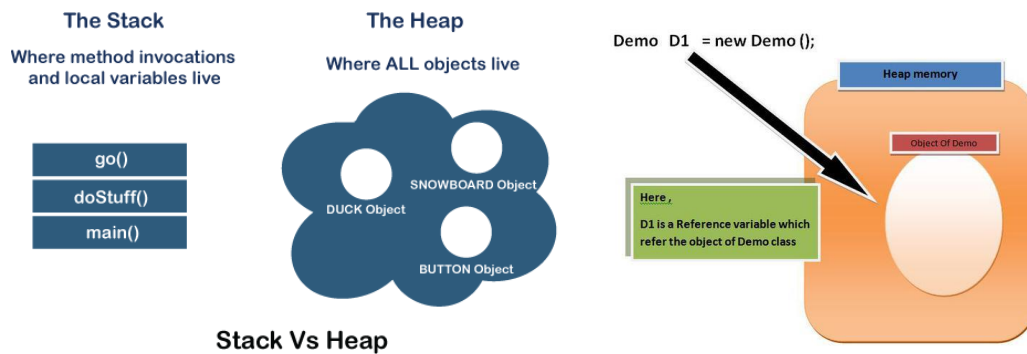
When we talk about "**objects**," we often mean instances of classes that are referred to using **reference variables**. However, the reference type in C++ is different than the reference type in Java.

Explore More...

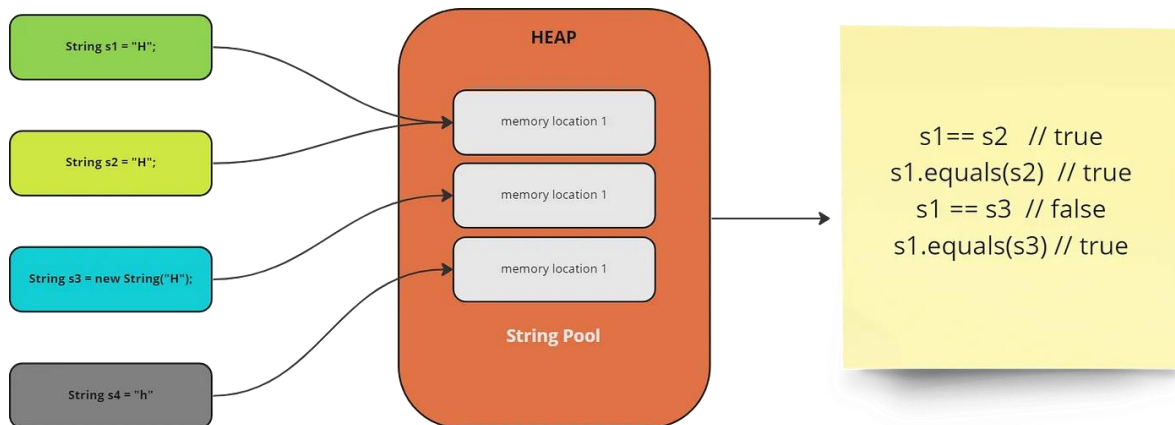
<https://www.geeksforgeeks.org/reference-variable-in-java/>

<https://www.tutorialspoint.com/C-Cplusplus-Pointers-vs-Java-references>

<https://www.geeksforgeeks.org/is-there-any-concept-of-pointers-in-java/>



String advanced



Immutable

```

/*
This code is created to demonstrate "String pool" in the heap memory. In addition,
it will also demonstrate the Immutable strings
Before Executing This code:
You must be aware of the Heap and Stack memory concept
*/

public class Demo_String {
    public static void main(String args[]){
        String s1 = new String("Nimra");
        String s2 = new String("Nimra");
        System.out.println(s1==s2); //This statement will return False

        //Without the new keyword
        s1 = "Nimra";
        s2= "Nimra";
        System.out.println(s1==s2); //This will return True

        //String are Immutable... Let's see
        System.out.println("Before append: " + s1 + ": " + s1.hashCode());
        s1 += " Mughal";
        System.out.println("After append: " + s1 + ": " + s1.hashCode());
        //Hashes are different before and after append

    }
}

```

Mutable strings (String Buffer and String builder)

```
public class Demo_StringBuffer {
    public static void main(String args[]){
        StringBuffer sb = new StringBuffer("Nimra");
        //To append another string/text
        System.out.println("Before append:"+ sb + ": " + sb.hashCode());
        sb.append(" Mughal");
        System.out.println("After append:"+ sb + ": " + sb.hashCode());
        System.out.println(sb.capacity());

    }
}
```

String Regular Expressions

```
import java.util.regex.*;
import java.util.*;
public class RegexExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Example 1: Checking if a String contains a specific pattern
        String text1 = "This is the sample text just to explore
https://www.finance.gov.pk/";
        String pattern1 = "[a-zA-Z0-9._%+-]+gov.pk";
        boolean containsPattern = Pattern.compile(pattern1).matcher(text1).find();
        System.out.println("Contains 'xyz@gov.pk': " + containsPattern);

        // Example 2: To check whether the email is valid or not
        // Regex pattern for validating email addresses
        String emailPattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$";
        // Input email to be validated
        System.out.println("Enter your email");
        String email = sc.nextLine(); // Replace with the email to be validated
        // Check if the email matches the pattern
        boolean isValidEmail = email.matches(emailPattern);
        System.out.println("The entered email is? :" + isValidEmail);

        // Example 3: Replacing parts of a String using regex
        String text3 = "The quick brown fox jumps over the lazy dog";
        String replacedText = text3.replaceAll("fox", "cat");
        System.out.println("Replaced text: " + replacedText);

    }
}
```

Input ways

JOptionPane

```
import javax.swing.JOptionPane;

public class JOptionPaneExample {
    public static void main(String[] args) {
        //String input(Default)
        String name = JOptionPane.showInputDialog("Enter your name:");
        JOptionPane.showMessageDialog(null, "Hello, " + name + "! Welcome.");

        //Integer Input
        String str = JOptionPane.showInputDialog("Enter Any Number: ");
        int num = Integer.parseInt(str);
        JOptionPane.showMessageDialog(null, "you entered" + num + "! Welcome.");

        //For float/double
        str = JOptionPane.showInputDialog("Enter Any Number: ");
        float num1 = Float.parseFloat(str);
        JOptionPane.showMessageDialog(null, "you entered" + num1 + "! Welcome.");
    }
}
```

Command Line Arguments

```
public class CMDArgumentsExample {
    public static void main(String[] args) {
        if(args.length > 0) {
            System.out.println("Arguments passed from command line:");
            for (String arg : args) {
                System.out.println(arg);
            }
        } else {
            System.out.println("No arguments provided.");
        }
    }
}
```

To run this code after compilation

```
java CMDArgumentsExample argument1 argument2 argument3
```

Output formatting

```
public class PrintfExample {
    public static void main(String[] args) {
        String name = "John";
        int age = 30;
        double height = 6.1;

        System.out.printf("Name: %s, Age: %d, Height: %.1f\n", name, age, height);
        System.out.println();
        // Format strings to be placed in fixed width
        System.out.printf("Name: %-10s, Age: %-5d, Height: %-5.1f\n", name, age,
height);

        double d = 4.5;

        System.out.println(d);
        // Shows 2 digits after the decimal point
        System.out.println(String.format("floatValue: %.2f", d)); // Shows 2 digits
after the decimal point
    }
}
```

Arrays in java

<https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html>

<https://www.hackerrank.com/challenges/java-1d-array-introduction/problem?isFullScreen=true>

<https://www.w3resource.com/java-exercises/array/index.php>

Jagged array (UnEven Array)

Each student is enrolled in a different number of subjects. A **jagged array** can store marks where each row represents a **student**, and the columns represent **marks in subjects**.

```
public class JaggedArrayExample {
    public static void main(String[] args) {
        // Jagged array: Different students have different subject counts
        int[][] studentMarks = {
            {85, 90, 78},          // Student 1: 3 subjects
            {92, 88},              // Student 2: 2 subjects
        };
    }
}
```

```

        {76, 81, 90, 85},    // Student 3: 4 subjects
        {80}                // Student 4: 1 subject
    };

    // Display student marks
    for (int i = 0; i < studentMarks.length; i++) {
        System.out.print("Student " + (i + 1) + " Marks: ");
        for (int mark : studentMarks[i]) {
            System.out.print(mark + " ");
        }
        System.out.println();
    }
}

```

Operators Precedence and Ternary Operator

<https://www.javatpoint.com/operators-in-java>

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html>

<https://www.refreshjava.com/java/operator-precedence>

<https://www.geeksforgeeks.org/java-ternary-operator-with-examples/>

Exercises

1. **Mega Exercise: Solve Exercises from the HackerRank from the covered topics (As many as you can. I have listed here two basic challenges)**

<https://www.hackerrank.com/domains/java>

<https://www.hackerrank.com/challenges/java-output-formatting?isFullScreen=true>

<https://www.hackerrank.com/challenges/java-1d-array-introduction/problem?isFullScreen=true>

2. **Generate random passwords:** Write a code that generates a random password of a specified length, using a combination of letters, numbers, and symbols.
3. **Create a class Person with fields for name, age, and address.**

- Use @author and @version tags in the class comment.
- Format comments with HTML tags for emphasis and lists (e.g.,).

4. Employee Weekly Working Hours

A company wants to track weekly working hours of employees.

- Each department has a different number of employees.
- Store 5 days of working hours per employee.
- Calculate the total and average working hours for each employee.

Input Example

```
Enter number of departments: 2
Enter number of employees in Department 1: 2
Enter hours for Employee 1: 8 9 8 9 8
Enter hours for Employee 2: 7 8 7 8 7

Enter number of employees in Department 2: 1
Enter hours for Employee 1: 6 7 6 7 6
```

Expected Output:

```
Department 1:
Employee 1: Total = 42, Average = 8.4
Employee 2: Total = 37, Average = 7.4

Department 2:
Employee 1: Total = 32, Average = 6.4
```

5. Password Validator: Write a Java code that can verify whether the string is a valid password or not

- a. Minimum 8 characters long
- b. Contains at least one digit
- c. Contains at least one alphabet

d. Contains at least one special character

6. Check for anagrams: Write a code that determines whether two strings are anagrams (have the same characters in different arrangements).

```
PS E:\Collaborative Learning\Spring_24_BS-II_OOP\Codes\lab3> java AnagramCheck
Enter any two strings
listen
silent
Strings "listen" and "silent" are anagrams.
PS E:\Collaborative Learning\Spring_24_BS-II_OOP\Codes\lab3> |
```

7. Solve Exercises of **Bitwise operators** from previous labs if you couldn't solve them earlier

8. Airport Flight Seating System

An airport has flights, and each flight has different seat arrangements.

- Store seat availability for each flight.
- Allow the user to book a seat.

```
Enter number of flights: 2
Enter number of seats in Flight 1: 4
Enter number of seats in Flight 2: 3

Available Seats:
Flight 1: [Available] [Available] [Available] [Available]
Flight 2: [Available] [Available] [Available]

Book a seat:
Enter Flight Number (1-2): 1
Enter Seat Number (1-4): 3
```

Expected Output

```
Flight 1: [Available] [Available] [Booked] [Available]
Flight 2: [Available] [Available] [Available]
```